



Aplicação de Lista de Compras com Preços e Alertas

SHOPISEL

Orientador:

Doutor Fernando Miguel Gamboa de Carvalho

Júri:

Presidente:

Vogais:

Doutor

Doutor

Julho de 2026

Aplicação de Lista de Compras com Preços e Alertas

SHOPISEL

Orientador:

Doutor Fernando Miguel Gamboa de Carvalho, IPL/ISEL

Júri:

Presidente:

Vogais:

Julho de 2026

Agradecimentos

Gostaríamos de expressar o nosso sincero agradecimento ao Doutor Fernando Miguel Gamboa de Carvalho pela sua disponibilidade constante e pela rapidez nas respostas ao longo do desenvolvimento deste projeto, contributos esses que se revelaram fundamentais para a sua concretização.

Agradecemos igualmente ao João Pereira, ao Pedro Tainha e ao Breno Nico pela idealização do projeto, bem como por todas as explicações, esclarecimentos e orientações fornecidas, que foram essenciais para a definição da estrutura, abordagem e formas de pensar adotadas ao longo deste trabalho.

A todos, o nosso muito obrigado pelo apoio e colaboração.

Declaração de integridade

Declaro que este trabalho de projeto é o resultado da minha investigação pessoal e independente. O seu conteúdo é original e todas as fontes listadas nas referências bibliográficas foram consultadas e estão devidamente mencionadas no texto. Mais declaro que todas as referências científicas e técnicas relevantes para o desenvolvimento do trabalho estão devidamente citadas e constam das referências bibliográficas.

O autor

Lisboa, 20 de Julho de 2026

Abstract

This project presents the development of a mobile application for managing shopping lists enhanced with price comparison and alert functionalities, supported by a scalable microservices-based backend architecture. The solution is designed to be used in real-world contexts, such as supermarkets, allowing users to efficiently create and manage shopping lists, mark items during purchases, and access updated product prices across different stores and locations.

The system integrates multiple technologies, including a React Native mobile application, a React-based web application for backoffice management, and a set of RESTful backend services developed in .NET. Authentication and authorization are handled através de Keycloak, while push notifications are delivered through Firebase Cloud Messaging to inform users about price changes and list updates.

A central data management component is responsible for handling products, prices, and store information, ensuring consistency and reliability. The architecture follows a modular and scalable approach, enabling the integration of additional services and supporting high performance and data synchronization requirements.

Additionally, the project explores the concept of White Label development, allowing the customization of applications through dedicated CI/CD pipelines. The use of AI-powered tools, such as GitHub Copilot, is also incorporated to support development and validation processes.

Overall, this work demonstrates how a seemingly simple application can evolve into a complete digital ecosystem, addressing challenges related to distributed systems, data management, and real-time user interaction.

Resumo

O presente projeto apresenta o desenvolvimento de uma aplicação móvel para gestão de listas de compras, enriquecida com funcionalidades de comparação de preços e definição de alertas, suportada por uma arquitetura de backend baseada em microserviços. A solução foi concebida para utilização em contextos reais, como supermercados, permitindo aos utilizadores criar e gerir listas de compras de forma eficiente, marcar itens durante a compra e consultar preços atualizados de produtos em diferentes lojas e localizações.

O sistema integra diversas tecnologias, incluindo uma aplicação móvel desenvolvida em React Native, uma aplicação web em React para backoffice e um conjunto de serviços backend com APIs REST implementados em .NET. A autenticação e autorização são asseguradas através do Keycloak, enquanto as notificações são geridas com recurso ao Firebase Cloud Messaging, permitindo alertar os utilizadores para alterações de preços e atualizações das listas.

A solução inclui ainda um mecanismo centralizado de gestão de dados, responsável pela manutenção de informação relativa a produtos, preços e lojas, garantindo consistência e fiabilidade. A arquitetura adotada segue princípios modulares e escaláveis, possibilitando a evolução do sistema e a integração de novos serviços.

Adicionalmente, o projeto explora o conceito de desenvolvimento White Label, permitindo a customização das aplicações através de pipelines dedicados de CI/CD. Foi também utilizada inteligência artificial, nomeadamente ferramentas baseadas em LLMs como o GitHub Copilot, como apoio ao desenvolvimento e validação do sistema.

Em suma, este trabalho demonstra como uma aplicação aparentemente simples pode evoluir para um ecossistema digital completo, abordando desafios relacionados com sistemas distribuídos, gestão de dados e interação em tempo real com o utilizador.

Contents

Agradecimentos	i
Declaração de integridade	iii
Abstract	v
Resumo	vii
Acronyms	xv
1 Introduction	1
1.1 Context and motivation	1
1.1.1 Scope of the Problem	2
1.2 Approach	2
2 State-Of-The-Art	5
2.1 Plataformas digitais para apoio a compras	5
2.2 Arquitetura orientada a serviços	5
2.3 Persistência de dados e modelo relacional	6
2.4 Aplicações web modernas	6
2.5 Aplicações móveis multiplataforma	7
2.6 Autenticação e autorização	7
2.7 Recolha automática de dados	7
2.8 Containerização, CI/CD e deployment	8
2.9 Síntese	8
3 Related Work	9
4 Developed Work	11
5 EMP and SEP Usage and Workflow	13
6 Testing and Evaluation	15
7 Conclusion	17

List of Figures

List of Tables

Acronyms

API Application Programming Interface

HTTP HyperText Transfer Protocol

JSON JavaScript Object Notation

JWT JSON Web Token

REST Representational State Transfer

Chapter 1

Introduction

O planeamento de compras continua a exigir que os consumidores consultem múltiplas fontes de informação para comparar preços, verificar promoções ou organizar listas de produtos. Este processo torna-se frequentemente demorado e ineficiente, especialmente quando envolve várias lojas ou supermercados.

Este projeto propõe o desenvolvimento de uma plataforma digital que centraliza informação de produtos e preços, permitindo aos utilizadores criar listas de compras inteligentes, comparar preços entre diferentes lojas e receber alertas quando ocorrem alterações relevantes.

A solução será disponibilizada através de duas interfaces principais: uma aplicação móvel e uma aplicação web.

A aplicação móvel será utilizada no contexto real de compras e permitirá aos utilizadores criar e gerir listas de forma simples, consultar e comparar preços entre diferentes lojas e visualizar preços próximos com base na localização do utilizador. Adicionalmente, os utilizadores poderão receber notificações quando um produto sofrer uma redução de preço e identificar rapidamente produtos através da leitura de códigos de barras ou códigos QR com a câmara do dispositivo.

A aplicação web permitirá uma preparação mais confortável das listas de compras num computador, bem como a consulta de comparações de preços entre diferentes lojas e localizações, oferecendo uma visão mais abrangente da informação disponível.

Um dos elementos centrais da plataforma será a georreferenciação de preços. O mesmo produto poderá ter valores diferentes consoante a loja e a localização geográfica, permitindo ao sistema apresentar ao utilizador as opções mais vantajosas com base na sua proximidade. Desta forma, a aplicação pretende simplificar o processo de planeamento de compras e ajudar os utilizadores a tomar decisões mais informadas, poupando tempo e dinheiro.

1.1 Context and motivation

Nos últimos anos, a necessidade de otimizar o processo de planeamento de compras tem vindo a aumentar, impulsionada pela crescente diversidade de produtos, lojas e promoções disponíveis. Atualmente, os consumidores são frequentemente obrigados a consultar múlti-

plas fontes de informação para comparar preços, verificar campanhas promocionais e organizar listas de compras, o que torna este processo moroso e pouco eficiente. Esta realidade é particularmente evidente quando envolve diferentes superfícies comerciais, como supermercados e lojas físicas, onde os preços podem variar significativamente.

Este projeto foca-se especificamente no desenvolvimento de uma plataforma digital integrada que permita centralizar informação de produtos e preços, simplificando o processo de tomada de decisão do utilizador. A solução proposta combina uma aplicação móvel e uma aplicação web, permitindo não só a criação e gestão de listas de compras, mas também a comparação de preços entre diferentes lojas e localizações. Adicionalmente, são introduzidas funcionalidades como alertas de alteração de preços, leitura de códigos de barras e georreferenciação, proporcionando uma experiência mais inteligente e adaptada ao contexto real de compra.

1.1.1 Scope of the Problem

A gestão eficiente de compras apresenta vários desafios que vão além da simples criação de listas de produtos. Um dos principais problemas reside na dispersão da informação, uma vez que os preços dos produtos variam consoante a loja e a localização geográfica, obrigando o utilizador a realizar múltiplas consultas para encontrar a melhor opção. Esta dificuldade é reforçada pelo facto de muitas soluções existentes se focarem apenas em contextos de comércio eletrónico, não respondendo adequadamente às necessidades de compras em lojas físicas.

Outro desafio relevante prende-se com a falta de integração entre funcionalidades. Aplicações atuais tendem a oferecer apenas gestão de listas ou comparação de preços de forma isolada, não existindo uma solução unificada que combine ambas de forma eficiente. Para além disso, a inexistência de mecanismos automáticos de alerta para variações de preço limita a capacidade do utilizador de aproveitar oportunidades de poupança em tempo útil.

Neste contexto, o problema a abordar passa pela criação de um sistema que integre, de forma coerente e escalável, a gestão de produtos, preços e lojas, garantindo simultaneamente desempenho, consistência de dados e relevância da informação apresentada ao utilizador. A introdução de georreferenciação e notificações em tempo real surge como elemento diferenciador, permitindo apresentar opções mais vantajosas com base na proximidade e nas preferências do utilizador.

1.2 Approach

Para alcançar os objetivos definidos, foi concebido e desenvolvido um sistema distribuído composto por uma aplicação móvel, uma aplicação web e uma infraestrutura de backend, recorrendo a tecnologias modernas amplamente adotadas, de forma a garantir usabilidade, escalabilidade e facilidade de manutenção.

A solução proposta segue uma arquitetura orientada a serviços, na qual o backend se

encontra dividido em vários serviços responsáveis por domínios específicos, como produtos, preços, lojas e utilizadores. Estes serviços são disponibilizados através de APIs REST, permitindo uma comunicação eficiente entre os diferentes componentes e promovendo uma clara separação de responsabilidades.

A aplicação móvel, desenvolvida com recurso a React Native, foi concebida para utilização em contexto real de compra, disponibilizando funcionalidades como gestão de listas, comparação de preços, leitura de códigos de barras e receção de notificações em tempo real. Por sua vez, a aplicação web, desenvolvida em React, oferece uma interface mais completa para preparação e análise das compras, tirando partido de um ambiente de utilização mais confortável.

A gestão de dados é assegurada através de uma base de dados relacional, garantindo integridade e eficiência nas operações de consulta, particularmente em cenários onde um mesmo produto pode apresentar preços distintos consoante a loja e a localização. A incorporação de mecanismos de georreferenciação permite ao sistema apresentar informação contextualizada, baseada na proximidade do utilizador.

Adicionalmente, o sistema integra um mecanismo de notificações push, possibilitando o envio de alertas relativos a alterações de preços, contribuindo para uma maior proatividade na tomada de decisão por parte do utilizador. O processo de desenvolvimento é suportado por pipelines de CI/CD, que permitem automatizar tarefas de construção, teste e deploy, assegurando a qualidade e consistência do software.

Em conjunto, esta abordagem permite o desenvolvimento de uma solução robusta, escalável e centrada no utilizador, capaz de responder de forma eficaz aos desafios identificados no domínio do planeamento de compras.

Chapter 2

State-Of-The-Art

Este capítulo apresenta o estado da arte das tecnologias, metodologias e ferramentas mais relevantes para o desenvolvimento da plataforma ShopISEL. A análise é orientada pelas necessidades concretas do projeto: disponibilizar uma solução composta por aplicação web, aplicação móvel, serviços de backend, persistência relacional, autenticação centralizada, recolha automática de dados de produtos e mecanismos de entrega contínua.

2.1 Plataformas digitais para apoio a compras

As soluções digitais para apoio a compras evoluíram de listas simples, mantidas localmente pelo utilizador, para plataformas integradas capazes de combinar catálogos de produtos, preços, histórico de alterações, alertas e informação contextual. No domínio dos supermercados, esta evolução é particularmente relevante, uma vez que o preço de um mesmo produto pode variar consoante a loja, a campanha promocional, a disponibilidade e a localização geográfica.

No contexto deste projeto, a plataforma não se limita a representar uma lista de compras. A solução pretende relacionar produtos, categorias, lojas, preços e utilizadores, permitindo que a lista seja um ponto de entrada para decisões de compra mais informadas. Esta abordagem aproxima a aplicação de sistemas de comparação e monitorização de preços, mas mantém o foco numa utilização prática e recorrente: preparar compras, consultar alternativas e acompanhar alterações relevantes ao longo do tempo.

2.2 Arquitetura orientada a serviços

Uma tendência consolidada no desenvolvimento de plataformas distribuídas é a separação do sistema em serviços especializados. Esta abordagem permite dividir responsabilidades por domínio funcional, reduzir o acoplamento entre componentes e facilitar a evolução independente de cada parte do sistema. No projeto ShopISEL, esta separação encontra-se refletida em serviços distintos para listas de compras, produtos, lojas e preços.

A utilização de Application Programming Interface (API)s Representational State Transfer (REST) continua a ser uma escolha frequente em sistemas web e móveis, devido à sua

simplicidade, interoperabilidade e boa integração com clientes baseados em HyperText Transfer Protocol (HTTP). Estudos recentes sobre frameworks para serviços web destacam que a escolha da tecnologia de backend deve considerar desempenho, escalabilidade, produtividade e maturidade do ecossistema [10]. No caso do ShopISEL, estes critérios justificam a adoção de serviços em ASP.NET Core, com endpoints organizados por domínio e contratos explícitos para a comunicação com as aplicações cliente.

O projeto utiliza Minimal APIs em ASP.NET Core, uma abordagem recomendada para a criação de APIs HTTP com menor quantidade de código estrutural e boa adequação a serviços pequenos ou focados num domínio específico [6]. Esta opção é coerente com a divisão do repositório, onde cada serviço encapsula o seu modelo de dados, endpoints, lógica de negócio e configuração de execução.

2.3 Persistência de dados e modelo relacional

A gestão de produtos, lojas, listas e preços exige consistência entre entidades relacionadas. Por esse motivo, as bases de dados relacionais continuam a ser uma solução adequada para sistemas onde existem relações fortes, restrições de integridade e necessidade de consultas estruturadas. PostgreSQL é uma das soluções relacionais mais utilizadas em aplicações modernas, sendo mantida com documentação oficial e suporte para versões atuais [9].

No ShopISEL, os serviços de backend recorrem a Entity Framework Core com o provider Npgsql para PostgreSQL. Esta combinação permite modelar entidades em C#, aplicar migrações e interagir com a base de dados através de uma camada de acesso a dados integrada no ecossistema .NET [7]. A utilização de migrações é particularmente importante porque o sistema evolui por serviços: cada domínio pode alterar o seu esquema de dados sem exigir uma reorganização global da aplicação.

Esta opção também facilita o suporte a cenários relevantes para o projeto, como a associação de itens a listas de compras, a categorização de produtos, o registo de lojas e a manutenção de informação de preços recolhida de fontes externas. A persistência relacional oferece, assim, uma base estável para relacionar dados operacionais com funcionalidades de comparação e consulta.

2.4 Aplicações web modernas

As aplicações web modernas privilegiam interfaces reativas, comunicação assíncrona com APIs e componentes reutilizáveis. React consolidou-se como uma biblioteca amplamente adotada para construção de interfaces declarativas, tirando partido de componentes e hooks para gerir estado, efeitos e composição de comportamentos [5]. Vite, por sua vez, é frequentemente utilizado em projetos frontend por simplificar a configuração e acelerar o ciclo de desenvolvimento.

No ShopISEL, a aplicação web é construída com React, TypeScript e Vite. Esta combi-

nação permite desenvolver uma interface orientada a componentes para consultar produtos, gerir listas, visualizar alertas e interagir com os serviços de backend. O uso de TypeScript acrescenta verificação estática ao código cliente, reduzindo erros em estruturas de dados trocadas com as APIs. A presença de bibliotecas de componentes e utilitários de interface, como Material UI, Radix UI e Tailwind, permite acelerar a construção de ecrãs consistentes sem abandonar a personalização visual da plataforma.

2.5 Aplicações móveis multiplataforma

No contexto de compras presenciais, a aplicação móvel é essencial porque acompanha o utilizador no local onde a decisão de compra acontece. Funcionalidades como consulta rápida de listas, leitura de códigos de barras, comparação de preços e receção de alertas dependem de uma experiência adaptada ao dispositivo móvel.

React Native permite desenvolver aplicações móveis com uma base tecnológica próxima do ecossistema React, enquanto Expo fornece ferramentas, módulos nativos e serviços que simplificam a criação de aplicações para Android, iOS e web a partir de um projeto JavaScript/TypeScript [2]. No repositório do ShopISEL, a aplicação móvel usa Expo, Expo Router, React Native, NativeWind e módulos como Expo Camera e Expo Secure Store. Esta escolha é adequada ao projeto porque permite integrar navegação por separadores, autenticação, armazenamento seguro e leitura por câmara, mantendo uma base de desenvolvimento coerente com a aplicação web.

2.6 Autenticação e autorização

Sistemas que manipulam listas pessoais e preferências de utilizador necessitam de autenticação fiável e de uma separação clara entre identidade e lógica aplicacional. Keycloak é uma solução de gestão de identidade e acesso que suporta protocolos como OpenID Connect e OAuth 2.0, permitindo proteger aplicações e serviços através de tokens e endpoints padronizados [4].

No ShopISEL, Keycloak é utilizado como componente central de autenticação. O serviço de listas valida tokens JSON Web Token (JWT) emitidos por Keycloak, verificando emissor, validade e partes autorizadas antes de permitir o acesso a recursos protegidos. Esta abordagem evita que cada serviço implemente autenticação própria e permite que web, mobile e backend partilhem um modelo comum de identidade.

2.7 Recolha automática de dados

Uma plataforma de comparação de preços depende da qualidade e atualização dos dados. Quando não existe uma API pública uniforme para todas as lojas, a recolha automática por

scraping torna-se uma alternativa prática, embora exija cuidados com estrutura das páginas, normalização de dados e robustez perante alterações externas.

O projeto inclui scrapers específicos para Continente e Pingo Doce. Estes componentes recolhem produtos por categoria, extraem informação como nome, preço, promoção, imagem e disponibilidade, e podem sincronizar os resultados com PostgreSQL através da variável de ambiente `PRODUCTS_DB_CONNECTION`. No caso do Pingo Doce, a utilização de Playwright é adequada a páginas dinâmicas, uma vez que esta ferramenta permite automatizar browsers e interagir com conteúdo renderizado no cliente. A existência de ficheiros JavaScript Object Notation (JSON) de saída também facilita a validação dos dados extraídos e a análise posterior por categoria.

2.8 Containerização, CI/CD e deployment

A entrega de aplicações compostas por vários serviços exige mecanismos que garantam reprodutibilidade e automatização. Docker permite empacotar aplicações e dependências em containers, tornando mais previsível a execução em ambientes distintos [1]. Esta característica é especialmente relevante no ShopISEL, onde coexistem serviços .NET, frontend web, gateway Nginx e componentes de suporte.

O repositório inclui Dockerfiles e workflows de GitHub Actions para automatizar validação, construção e publicação de imagens. GitHub Actions suporta a automatização de workflows diretamente no repositório, incluindo tarefas de integração e entrega contínua [3]. A configuração de deployment usa Render, com serviços baseados em imagens e um Nginx configurado como ponto de entrada para encaminhar pedidos para frontend, listas, produtos, lojas e preços. A documentação do Render suporta este modelo de deployment com imagens Docker pré-construídas ou Dockerfiles do repositório [8].

2.9 Síntese

O estado da arte analisado mostra que a arquitetura escolhida para o ShopISEL está alinhada com práticas atuais de desenvolvimento de aplicações distribuídas. A separação em serviços facilita a evolução por domínio, PostgreSQL e Entity Framework Core oferecem uma base consistente para persistência, React e Expo permitem cobrir web e mobile com tecnologias próximas, Keycloak centraliza autenticação e Docker com GitHub Actions apoia a entrega contínua.

Estas escolhas tecnológicas respondem diretamente aos objetivos do projeto: centralizar informação de produtos e preços, suportar listas de compras inteligentes, permitir comparação entre lojas e preparar a plataforma para evolução futura com novas fontes de dados, novas funcionalidades de alerta e melhorias na experiência do utilizador.

Chapter 3

Related Work

This chapter will explore two primary approaches to ...

Chapter 4

Developed Work

This chapter presents the architecture, technology stack, and implementation of the ...

Chapter 5

EMP and SEP Usage and Workflow

This chapter provides a detailed walkthrough of the ...

Chapter 6

Testing and Evaluation

The testing and evaluation phase is a key part of the development process, as it demonstrates the system's functionality, reliability, and security. It confirms the system's ability to perform as intended and identifies potential vulnerabilities.

This chapter details the testing methodology applied to ...

Chapter 7

Conclusion

This chapter summarises the problem addressed in this thesis and the proposed solution, providing an overview of the development...

Bibliography

- [1] Docker (2026). What is docker? <https://docs.docker.com/get-started/docker-overview/>. Accessed: 2026-04-27.
- [2] Expo (2025). Introduction to expo. <https://docs.expo.dev/get-started/introduction/>. Accessed: 2026-04-27.
- [3] GitHub (2026). Github actions documentation. <https://docs.github.com/en/actions/>. Accessed: 2026-04-27.
- [4] Keycloak (2026). Securing applications and services with openid connect. <https://www.keycloak.org/securing-apps/oidc-layers>. Accessed: 2026-04-27.
- [5] Meta Open Source (2026). Built-in react hooks. <https://react.dev/reference/react/hooks>. Accessed: 2026-04-27.
- [6] Microsoft (2025). Apis overview: Minimal apis. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis/overview>. Accessed: 2026-04-27.
- [7] Npgsql (2026). Npgsql entity framework core provider. <https://www.npgsql.org/efcore/>. Accessed: 2026-04-27.
- [8] Render (2026). Docker on render. <https://render.com/docs/docker>. Accessed: 2026-04-27.
- [9] The PostgreSQL Global Development Group (2026). Postgresql documentation. <https://www.postgresql.org/docs/>. Accessed: 2026-04-27.
- [10] Zanevych, O. (2024). Advancing web development: A comparative analysis of modern frameworks for rest and graphql back-end services. *Grail of Science*, (37):216–228.

